

Android Code Challenge

Welcome to this coding challenge! The goal is to assess your ability to build a structured, maintainable, and user-friendly Android application using Kotlin. Take this as an opportunity to demonstrate not just your technical skills, but also your problem-solving approach, attention to detail, and architectural decisions.

Task:

Build a native Android app using Kotlin that:

1. Fetches the JSON from the endpoint:
<https://gist.githubusercontent.com/aruana-lumiform/383e1291df3e0cc1d49aeb14d45f5b94/raw/lumiform-android-test.json> simulating an API request.
2. Displays the JSON content in a visually structured way, clearly representing the hierarchy and relationships between items.

The JSON follows this structure:

The JSON follows this structure:

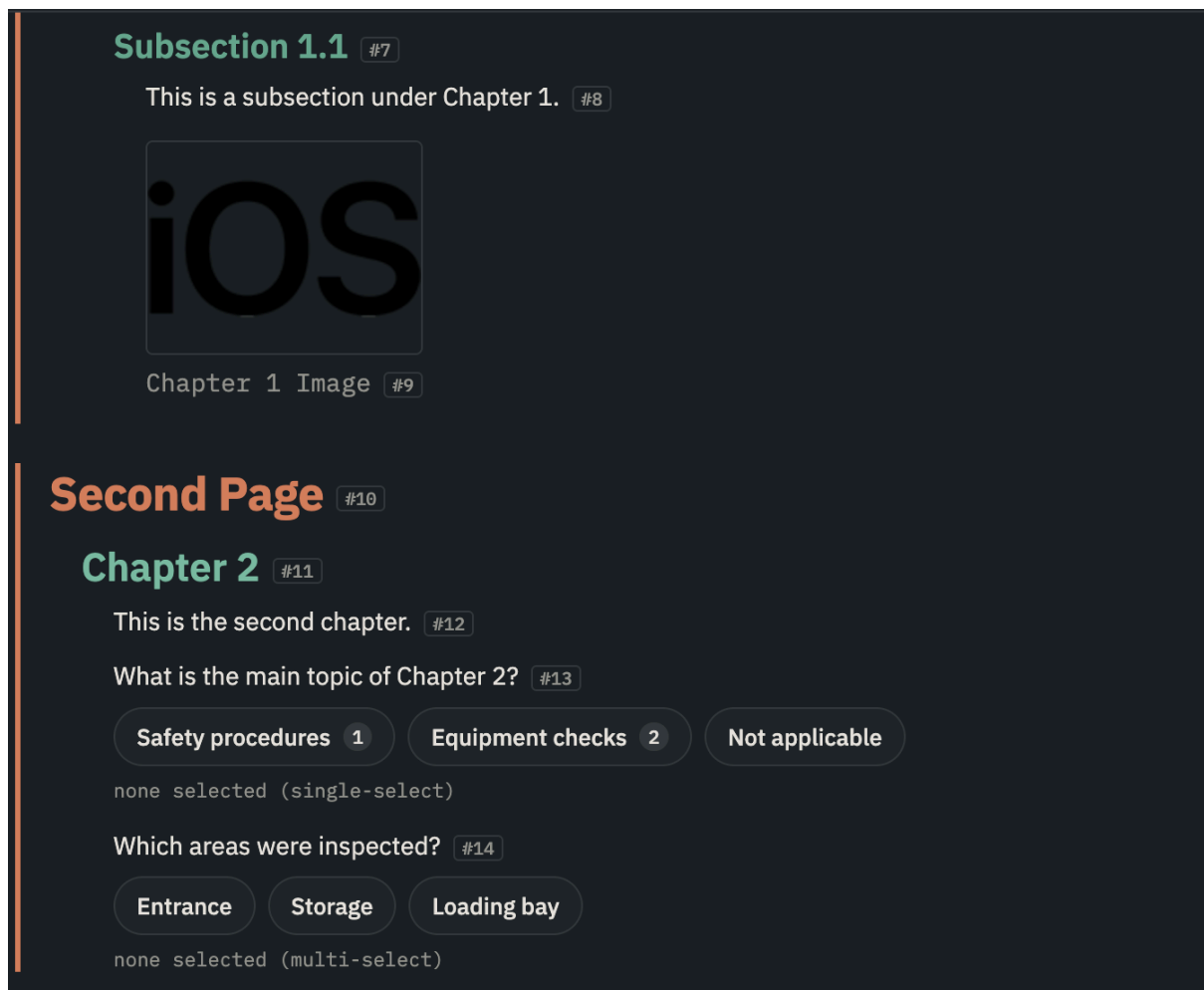
- It is a list of Pages, the topmost elements. Every item has a numeric id and a type.
- Pages contain Sections or Questions.
- Sections have a title and may contain nested Sections and Questions.
- Questions can be of type text, image, or choice.
- A choice question has a response_set listing the selectable responses. Its multiple_selection flag indicates whether more than one response may be selected at a time.

Requirements:

- Font sizes should reflect hierarchy:
 - Pages → Largest font size
 - Sections → Medium font size, decreasing for nested sections
 - Questions → Smallest font size
- For text-based questions, display the text directly.
- For image-based questions:
 - Fetch and display the image in a reduced size.

- Clicking the image should open a new screen displaying the full-sized image and its title.
- Implement offline support, ensuring the app can display previously fetched data. Do this using a relational database of your choice.
- For choice questions, display the responses from the `response_set` as selectable options, and respect `multiple_selection` when handling taps.

In the image below you can see a small example on how the screen might look like. This is just an example, feel free to use your own designs and interpretations.



Bonus (Optional):

- Handle network failures gracefully, providing a fallback mechanism for poor connections.

Feel free to leverage any online resources, including AI tools. However, keep in mind that your code quality, architecture, testing strategy, and ability to justify design choices will be evaluated.

Additional Instructions

- The app should be hosted in a public GitHub repository with a well-structured README containing clear setup instructions.
- A well-organized commit history is a plus, as it reflects good development practices.

Final Words

 This is your chance to showcase your expertise! Approach the challenge with creativity and attention to detail. Most importantly, enjoy the process and build something you're proud of!  